

How LLM-Assisted Coding Affects Functional Correctness

Kamlesh Kumar

Department of Computer Science

University of Helsinki

Helsinki, Finland

kamlesh.kumar@helsinki.fi

Abstract—LLM-assisted coding tools such as GitHub Copilot and ChatGPT are now widely used in software development, and their output is increasingly merged into production codebases. This paper focuses on one quality attribute only: functional correctness. The main concern is that LLM-generated or LLM-refactored code may introduce subtle logic errors, incorrect assumptions, or missed edge cases that are not immediately visible during review. The study uses a focused literature review of empirical software engineering research to examine how LLM-assisted coding affects correctness compared to non-LLM development. The goal is to identify the kinds of correctness problems most commonly reported, the contexts in which they appear, and the overall evidence on whether LLM use improves or degrades correctness. A secondary concern, which turns out to matter more than expected, is how the evaluation methodology itself — test-suite strength, non-determinism, and sampling strategy — shapes the correctness numbers that get reported. To keep the scope manageable, the paper excludes security, maintainability, and broader software quality concerns unless they directly relate to functional correctness. The expected outcome is a clear synthesis of current evidence on correctness-related risks and benefits in LLM-assisted coding.

Index Terms—large language models, code generation, functional correctness, empirical software engineering, literature review, GitHub Copilot

I. INTRODUCTION

Large language model (LLM) based coding tools have become part of everyday software development. Tools such as GitHub Copilot and ChatGPT are used by millions of developers to write, complete, and edit source code. Their output often reaches production codebases through normal pull-request flows. This raises a simple but important question for software engineering research: does the code produced with LLM assistance actually work?

This paper studies one specific quality attribute, *functional correctness*. By functional correctness we mean the extent to which a piece of code produces the outputs specified by its task description or test cases. Other attributes such as security, maintainability, and performance are also important, but they fall outside the scope of this review. Security has been studied in detail elsewhere [1] and combining it with correctness would blur the analysis.

The motivation to focus on correctness is practical. A correctness bug can cause a program to compute the wrong answer, crash in production, or silently corrupt data. Empirical work has shown that LLM-generated code sometimes fails in

ways that are not obvious during review, including missed edge cases, wrong assumptions about library behavior, and subtle logic mistakes [10], [11]. At the same time, widely cited benchmarks often report pass rates above 80% on simple Python problems [5]. These two pictures need to be reconciled — or at least set against each other in a way that clarifies when each is the more relevant one — and that is what this review tries to do.

The paper is organized around three research questions:

- **RQ1:** What does empirical evidence report about the functional correctness of code produced with LLM-assisted coding, and how does it compare to non-LLM development in the subset of studies where such a comparison is made?
- **RQ2:** Which factors are most consistently associated with variation in functional correctness, in particular programming language, prompt formulation, available context, task difficulty, and dependency complexity, and what recurring correctness failure modes do these factors produce?
- **RQ3:** What does the empirical literature report about how evaluation methodology itself, such as test suite strength, non-determinism, and number of generated candidates examined, affects measured functional correctness?

The remainder of the paper is organized as follows. Section II covers background. Section III describes how the literature was collected. Section IV synthesizes the findings for each research question. Section V discusses the overall picture, and Sections VI and VII cover threats to validity and conclusions.

II. BACKGROUND

LLM-assisted coding. We use this term to mean any software development activity in which a large language model generates, completes, or modifies source code. Common tools at the time of writing are GitHub Copilot, ChatGPT, Claude, and integrated development environment (IDE) plug-ins that call these models. The boundary between a tool and its underlying model matters less for this review than the basic behavior: a prompt goes in, code comes out.

Functional correctness. A piece of code is functionally correct if it satisfies the behavior described by a specification, typically a docstring plus a set of test cases. Most studies measure correctness through automated tests. The standard

metric is pass@k, which estimates the probability that at least one of k generated samples passes all tests [2].

Common benchmarks. Several benchmarks recur throughout the reviewed literature and are worth introducing briefly. HumanEval [2] is a set of 164 hand-written Python programming problems with reference test cases, introduced by OpenAI alongside Codex. MBPP (Mostly Basic Python Problems) is a similar but larger collection of crowd-sourced introductory Python tasks. Both have become de facto standards for reporting pass@k figures, and several of the studies in this review use one or both as a reference point. HumanEval+ [5] extends HumanEval with substantially stronger test suites. ClassEval [8] moves from function-level to class-level Python tasks. BigCodeBench [7] adds realistic library use across 139 Python libraries, and SWE-bench [6] evaluates models on real GitHub issues. The progression — from short self-contained functions toward realistic project work — becomes important in Section IV-A.

Benchmark vs. developer-in-the-loop evaluation. The literature splits into two streams. Benchmark studies run the model on a fixed set of self-contained problems and count how many pass. Developer-in-the-loop studies put real people in the picture through controlled experiments, field studies, or repository mining. Both streams answer different questions, and both are needed to understand what LLM assistance actually does to correctness.

Test-suite strength. Correctness is measured against tests, so the tests themselves matter. A test suite with only a handful of assertions per problem can miss many real bugs, which makes measured pass rates look higher than they really are. This point becomes important in Section IV-C.

Model capability vs. tool effect. Studies in this area evaluate two related but distinct objects. Some measure the underlying model on a fixed benchmark using direct API access (for example, GPT-4 on HumanEval). Others measure a tool that wraps a model with retrieval, ranking, IDE context, and post-processing (for example, GitHub Copilot inside an editor). These two settings produce different evidence: model-only evaluations isolate raw generation capability, while tool evaluations capture the full delivered behavior including any context the tool injects. In the rest of this paper we treat both as relevant evidence about correctness in deployment, but flag the distinction where it materially affects the interpretation of a reported number.

III. METHODOLOGY

This section describes how the literature was collected. We did not carry out a full PRISMA-style systematic literature review. The scope of a seminar paper does not justify that level of process, and the topic moves too quickly for a strict replication-ready protocol to be worthwhile. Instead we followed a focused literature review guided by the course’s mini-SLR material [4] and the reporting advice of Kitchenham et al. [3].

A. Search Strategy

We searched Scopus and IEEE Xplore, and used Google Scholar for recent work. The main search window was January 2023 through February 2026. We also included one paper from 2022 [14] because it is one of the earliest empirical evaluations of GitHub Copilot and is still heavily cited. The search string combined three concept groups with boolean AND: model or tool terms (“large language model,” “LLM,” “GPT,” “ChatGPT,” “Copilot,” “Codex,” “Claude”), task terms (“code generation,” “code completion,” “program synthesis,” “AI-assisted coding”), and quality terms (“correctness,” “functional correctness,” “bug,” “defect,” “pass@k,” “benchmark”). We also did one round of backward snowballing starting from EvalPlus [5], ClassEval [8], and the Dakhel et al. Copilot study [11], and one round of forward snowballing through Google Scholar’s “cited by” function.

B. Inclusion and Exclusion Criteria

The criteria were kept simple and single-purpose. For a paper to be included, all inclusion criteria had to be met and no exclusion criterion had to apply.

Inclusion: (I1) the paper presents empirical evidence about code produced with LLM assistance, (I2) it reports at least one measure related to functional correctness (pass rates, bug counts, test outcomes, human error rates), (I3) it is published between January 2023 and February 2026, or is a seminal 2022 work still heavily cited, and (I4) it is written in English.

Exclusion: (E1) the paper focuses exclusively on security, maintainability, or performance without reporting correctness evidence, (E2) it is a position or opinion piece without empirical data, (E3) it is a short poster or extended abstract without enough methodological detail, or (E4) it is a model technical report from a vendor with no independent evaluation.

C. Screening Procedure

Screening was performed iteratively rather than as a strict PRISMA-style flow with tracked counts at each stage. As candidate papers were encountered through database search and snowballing, we evaluated each against the inclusion and exclusion criteria and either retained, rejected, or held the paper for further reading at that point. We did not maintain explicit counts of records retrieved, after deduplication, or excluded by reason. This is a known limitation of focused reviews and is acknowledged again in Section VI. The choice was deliberate: a strict PRISMA flow is appropriate for a full systematic review, while a focused mini-SLR [4] optimizes for depth of synthesis on a small, carefully chosen corpus rather than for replication-grade process documentation. A second screener was not used; this is also acknowledged in Section VI.

D. Resulting Corpus

The final corpus contains eleven primary studies, summarized in Table I. Six are benchmark studies and five are developer-in-the-loop studies. Ten are peer-reviewed papers at recognized software engineering or machine learning venues.

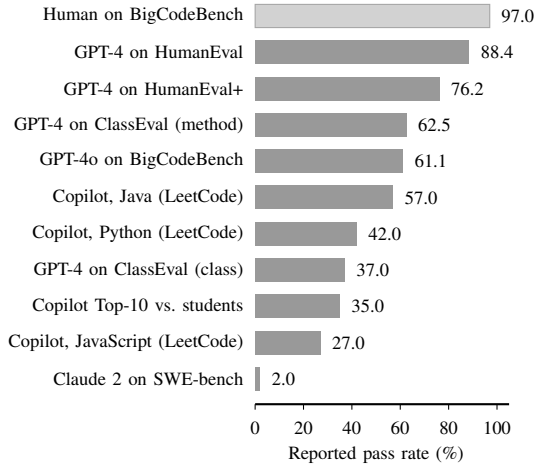


Fig. 1. Reported pass rates across the reviewed studies. The light bar at the top (P3) is the human baseline on BigCodeBench, shown for reference. All other bars are LLM results. The drop from simple standalone problems to realistic or dependency-heavy tasks is consistent and large.

One paper [14] is a short paper from MSR 2022, kept for its seminal role. The corpus was kept intentionally small to support depth of engagement with each primary study over breadth of coverage. We judged that a well-characterized small corpus better supports the evaluative claims developed in this paper than a larger corpus with shallower treatment per study.

E. Evidence Weighting

Following the course guidance on evidence strength, we did not treat every study as equally strong. When synthesizing the results we consider venue, sample size, study design, and whether the authors had a vested interest. Where a claim rests on a single study we note this, and where two or more studies converge we note the convergence. This is applied throughout Section IV.

IV. RESULTS

A. RQ1: Reported Correctness and Comparison with Non-LLM Development

Evidence for this question comes in two layers. Benchmark studies report what models can do on standalone problems, and developer-in-the-loop studies report what happens when real people use these tools. Fig. 1 summarizes the benchmark side at a glance.

Benchmark pass rates. On the original HumanEval benchmark, GPT-4 reaches around 88% pass@1. That number drops sharply once the benchmark is made harder. EvalPlus [5] augments HumanEval test suites by roughly 80 times on average, and GPT-4’s pass@1 falls from 88.4% to 76.2%. Across the 26 models that the EvalPlus authors studied, the reductions reached as high as 28.9 percentage points at pass@100. The authors also report that 11% of the ground-truth solutions shipped with HumanEval were themselves incorrect, which is a worrying finding about the benchmark many papers have been trusting as a baseline.

On benchmarks that require more context or more code, the picture is considerably less flattering. BigCodeBench [7] requires correct use of 139 Python libraries; GPT-4o, the best model the authors evaluated, reaches 61.1% on BigCodeBench-Complete compared to 97% for human programmers. ClassEval [8] adds another dimension by asking models to generate whole Python classes with interdependent methods rather than isolated functions — GPT-4 achieves 62.5% at the method level but only 37.0% at the class level on the same tasks, a drop large enough to matter for any practical use of these tools. Then there is SWE-bench [6]. Real GitHub issues from Django, scikit-learn, and similar large projects. Claude 2 resolved 1.96% of the 2,294 issues.

Cross-language results are broadly consistent across the two studies that measure them. Nguyen and Nadi [14] evaluated Copilot on 33 LeetCode problems in four languages and report Java 57%, JavaScript 27%, Python 42%, and C 39%. MultiPL-E [9], which translated HumanEval into 18 additional languages, shows that pass rates correlate with how often the language appears in training data. The authors also report that JavaScript sometimes matches or beats Python, which complicates the simple training-frequency story somewhat, and that adding type annotations to TypeScript does not produce a statistically significant improvement ($p = 0.33$). That last finding is a useful corrective to the common assumption that richer type information automatically helps the model, though it should be noted the result applies to one benchmark and one language and should probably not be over-extended.

Comparisons with non-LLM development. Direct comparisons between LLM-assisted and non-LLM development are fewer, and the results are mixed. Dakhel et al. [11] compared Copilot to a dataset of 4,225 student submissions across five Python tasks. Students had an overall correct ratio of 59%. Copilot’s Top-10 average was 35%, while its Top-1 suggestion was 52%. The authors summarize Copilot as an “asset for experts, liability for novices.” Their study also reports severe non-determinism: bubble sort was 100% correct in one trial and 0% correct in another trial run 30 days later, with no change in the prompt.

Perry et al. [12] ran a randomized controlled trial with 47 participants on five security-related programming tasks. The primary finding is about security rather than pure functional correctness, but participants using the AI assistant produced less correct code on four of the five tasks (for example, 67% vs. 79% on the encryption task). The authors also report a striking over-confidence effect: participants using the AI assistant believed their code was more secure than it actually was, a finding about confidence calibration that the study’s design was not set up to probe further.

Barke et al. [13] is qualitative rather than quantitative. The authors observed 20 programmers using Copilot on four tasks and identified two interaction modes, which they call acceleration (the programmer already knows what to do and uses Copilot for speed) and exploration (the programmer is unsure and uses Copilot to explore options). The paper reports no pass rates, but makes a point that is relevant to correctness: large

TABLE I
PRIMARY STUDIES INCLUDED IN THE REVIEW

ID	Short title	Venue	Study type	Language	Main correctness finding
P1	EvalPlus [5]	NeurIPS 2023	Benchmark, 26 models	Python	GPT-4 drops from 88.4% to 76.2% with stronger tests
P2	SWE-bench [6]	ICLR 2024	Benchmark, 2,294 issues	Python	Claude 2 resolves 1.96% of real GitHub issues
P3	BigCodeBench [7]	ICLR 2025	Benchmark, 1,140 tasks	Python	GPT-4o reaches 61.1% vs. 97% human performance
P4	ClassEval [8]	ICSE 2024	Benchmark, 100 classes	Python	GPT-4 drops from 62.5% (method) to 37.0% (class)
P5	MultiPL-E [9]	IEEE TSE 2023	Benchmark, 19 languages	19 languages	Training-data frequency correlates with pass rate
P6	Copilot robustness [10]	ICSE 2023	Empirical, 892 methods	Java	46% of paraphrased prompts yield different code
P7	Copilot: asset/liability [11]	JSS 2023	Empirical comparison	Python	Students 59% correct vs. Copilot Top-10 35%
P8	AI assistants RCT [12]	CCS 2023	RCT, 47 participants	Py, JS, C	AI users produced less correct code, over-confident
P9	Grounded Copilot [13]	OOPSLA 2023	Qualitative, 20 participants	Multiple	Two use modes. Verifying suggestions is hard
P10	Copilot empirical [14]	MSR 2022	Empirical, 33 problems	C, Java, JS, Py	Java 57%, Py 42%, C 39%, JS 27% pass rate
P11	Prompt programming [15]	IEEE TSE 2025	Factorial exp., 7,072 prompts	Python	Best/worst prompt gap is only around 10 pp

Copilot suggestions were harder for programmers to verify than code they had written themselves, and one participant failed to complete a task because they did not notice that Copilot had opened a file read-only.

Taken together, the evidence paints a less flattering picture than the headline numbers suggest. Pass rates on simple benchmarks are high, but the same models struggle on anything that requires real context. The comparisons with non-LLM development are limited in number and mixed in outcome — none of the studies in this corpus provides strong evidence that LLM assistance actually improves measured functional correctness. It is easy to interpret faster code output as better code output, but that inference is not supported by what the reviewed studies actually show.

B. RQ2: Factors Affecting Correctness and Recurring Failure Modes

Programming language. Nguyen and Nadi [14] and MultiPL-E [9] both find that language frequency in the training data correlates with pass rate. Well-represented languages such as Java and Python tend to do better than C. The MultiPL-E authors add the TypeScript finding already noted above: static type annotations did not produce a statistically significant improvement ($p = 0.33$ on HumanEval), which complicates the intuition that stronger type systems should help.

Prompt formulation. Khojah et al. [15] ran a full factorial experiment with five prompt techniques, three models, and 221 Python tasks. The gap between the best combination (function signature plus few-shot examples) and the worst (chain-of-thought alone) on GPT-4o is about 10 percentage points. The practical interpretation of this gap is mixed, it is large enough to be worth knowing about, but small enough that

it will not rescue a poorly specified task. Only two techniques produced statistically significant improvements: providing a function signature, and few-shot examples. Chain-of-thought, persona framing, and package listing did not significantly improve correctness, which is worth noting given how much attention chain-of-thought in particular has received in the broader prompting literature.

Mastropaolo et al. [10] approach the question from the other direction. They paraphrased 892 Java method descriptions without changing their meaning, and found that Copilot produced different code in about 46% of cases. Even when both versions passed the tests, 38 out of 136 test-passing predictions could only be obtained through one phrasing or the other. The implication — and Mastropaolo et al. are appropriately careful not to over-generalize from a single study — is that the model is sensitive to surface wording in ways that have nothing to do with the underlying task.

Task difficulty and dependency complexity. This is the strongest factor in the reviewed literature, and it is worth pausing on what the numbers actually show. ClassEval [8] shows a clear drop from function-level (62.5%) to class-level (37.0%) generation. BigCodeBench [7] shows that library dependency complexity alone cuts GPT-4o’s performance well below what it achieves on HumanEval. SWE-bench [6] shows that repository-level tasks with cross-file edits remain hard even for frontier models. The common thread across all three of these results is that context is the limiting factor — and not just any context, but the kind of dense, cross-file, project-specific context that characterizes real software engineering work rather than competitive programming exercises. The gap between HumanEval performance and real-issue performance

is not gradual; a model that scores near the ceiling on the former can be nearly useless on the latter.

Recurring failure modes. The corpus does not contain a single large failure-mode taxonomy, but several studies identify the same kinds of bugs. Nguyen and Nadi [14] report that Copilot frequently generates calls to helper functions it never implements, and in one case produced a Python solution with 43 chained `if` statements (cyclomatic complexity 43). Mastropaolo et al. [10] show cases where paraphrased descriptions caused Copilot to produce code that passed tests but implemented different behavior — for example returning a three-dimensional point instead of a two-dimensional one. Barke et al. [13] report that programmers frequently struggled to verify Copilot’s larger suggestions during review. These observations cluster around a small set of patterns: hallucinated helpers, silent changes in behavior after paraphrasing, and failures that look plausible enough to pass review. That last category is arguably the most consequential, precisely because it is the hardest to catch. Prompt formulation, programming language, and training-data distribution all contribute to correctness outcomes, but none of them comes close to the explanatory power of task difficulty and context complexity.

C. RQ3: How Evaluation Methodology Affects Measured Correctness

Measured correctness is not only a property of the code. It is also a property of how the measurement is done. Three methodology factors emerge from the corpus.

Test-suite strength. This is the best-documented factor. EvalPlus [5] increases the number of test cases on HumanEval by roughly 80 times and reports a drop of 19 to 29 percentage points across the 26 models it evaluates. More importantly, the ordering of models changes: two open-source models that looked no better than ChatGPT on HumanEval actually outperformed it on HumanEval+. The authors also found that 11% of the ground-truth solutions shipped with HumanEval were themselves incorrect. ClassEval [8] makes a related point by design — its test suites have 98.2% branch coverage and 83.7% mutation score, which are substantially higher than HumanEval and MBPP. The fact that pass rates on ClassEval are lower than on HumanEval reflects both harder tasks and stronger tests, and disentangling the two contributions is not straightforward.

Generation strategy and non-determinism. ClassEval [8] also shows that the way the code is generated affects measured pass rates. Generating a whole class at once is the best strategy for GPT-4 and GPT-3.5 but not for weaker models, which means a single “pass rate” number hides a methodological choice. Dakhel et al. [11] provide a concrete warning about non-determinism: their bubble-sort evaluation was 100% correct in one trial and 0% correct in another trial conducted 30 days later with no change in the prompt. This is a striking result for a paper published at a major software engineering venue, and it has received limited attention in subsequent work. Khojah et al. [15] address non-determinism more carefully by running each of their 7,072 prompt combi-

TABLE II
FACTORS ASSOCIATED WITH VARIATION IN MEASURED CORRECTNESS

Factor	Type	Effect on correctness	Evidence
Task difficulty / dependency complexity	Task	Largest drop (e.g., 88% $\rightarrow$ 2%)	P1, P2, P3, P4
Programming language	Code	Tracks training-data frequency	P5, P10
Prompt formulation	Input	Small ($\sim$ 10 pp gap)	P11, P6
Type annotations	Input	Not statistically significant	P5
Test-suite strength	Method	−19 to −29 pp on HumanEval+	P1, P4
Generation strategy	Method	Re-orders models	P4
Non-determinism	Method	Range up to 0% $\rightarrow$ 100% in one task	P7, P11
Choice of k in pass@ k	Method	Tens of pp difference	P1, [2]

nations three times. Most benchmark studies in the corpus use only one or two runs per task, which may well be too few to produce stable numbers.

Sampling and pass@ k . The choice of k in pass@ k has a large effect on reported numbers. Even within one paper and one model, pass@1 and pass@100 can differ by tens of percentage points [2]. A pass@100 result means the model can produce correct code if the test is run 100 times and the right one is picked. That is a legitimate measurement, but it is not what a developer actually experiences when accepting the first suggestion. Perplexity is sometimes used as a cheap proxy for capability, but MultiPL-E [9] reports that perplexity does not correlate strongly with pass@1 across languages, which means it cannot substitute for execution-based evaluation and probably should not be treated as a rough equivalent.

A single pass@1 figure, without knowing the test suite and the number of runs behind it, carries less information than it appears to. Stronger tests lower it, different generation strategies move it, and non-determinism adds noise that single-run evaluations will silently absorb — all of which is to say that comparing pass@1 figures across papers requires knowing quite a lot about each paper’s setup, a point the discussion returns to.

Table II summarizes the factors discussed in Sections IV-B and IV-C, separating those that belong to the code or task itself from those that belong to the measurement procedure.

V. DISCUSSION

A. Overall Picture

Three broad observations come out of the reviewed evidence. First, the gap between headline benchmark numbers and realistic task performance is consistent and large. On HumanEval the best models are near the ceiling. On Big-CodeBench, ClassEval class-level, and SWE-bench they are

much further from it, and a reader who only encounters the headline numbers will substantially overestimate reliability. Second, the comparisons with non-LLM development show that the productivity gains and correctness gains are not the same thing — Dakhel et al. and Perry et al. both find that LLM assistance did not improve correctness in their settings, and Perry et al. add the troubling finding that users became more confident precisely when their code was less correct. Third, the dominant factor controlling measured correctness is task difficulty and dependency complexity. The more a task needs project-specific context, the worse the models do. How steep that drop is remains somewhat benchmark-dependent, but the direction is consistent across every study that addresses it.

What this adds up to is that LLM assistance shifts the distribution of correctness outcomes, not just the average. On simple, self-contained, well-specified tasks, the code is usually correct. On complex, context-rich, or under-specified tasks, it is less reliable, and the mistakes tend to be subtle rather than obvious. That asymmetry is what makes review and verification important even when the average pass rate looks reasonable. Whether the over-confidence effect from Perry et al. generalizes is harder to say — 47 participants on security-focused tasks is a thin basis for a strong claim, and we do not lean heavily on it alone — but it is consistent enough with what Barke et al. observe qualitatively that dismissing it entirely seems unwarranted.

It is also worth restating the model-versus-tool distinction introduced in Section II when interpreting these results. The strongest comparison-with-humans evidence in the corpus comes from studies of GitHub Copilot [11]–[13], which packages a base model with retrieval and IDE context. The cleanest benchmark-only numbers come from API evaluations of base models. Where the two diverge — for example, when a tool with retrieval performs differently than a base model on the same task — the divergence is itself evidence about what the tool layer adds. The corpus does not contain enough head-to-head comparisons to quantify this difference systematically, which is itself a useful gap to flag for future work.

B. Practical Implications

The most immediate implication is about test suites. The EvalPlus result is difficult to set aside: passing a weak test suite does not mean the code is correct, it means it passed a weak test suite. For any team deploying LLM-generated code in production, tests need to be treated as a measurement instrument rather than just a regression safety net, and they need to be strong enough to actually surface the edge-case failures that LLMs produce. A team that trusts HumanEval-level coverage on LLM-generated code is, at minimum, not measuring what it thinks it is measuring.

The second implication concerns scope. The pass-rate cliff between function-level and class-level generation, and especially the SWE-bench numbers, suggest that using LLM output as a first-pass solution on complex, context-dependent tasks is a gamble that the current evidence does not favor. That does not mean the tools are useless in those settings — Barke

et al. show that programmers use them productively in an exploratory mode — but the output needs proportionally more scrutiny, not less, exactly because the failure modes there are harder to spot.

The third finding worth flagging is the over-confidence effect from Perry et al. If developers cannot reliably tell when LLM-assisted code is worse than what they would have written themselves, that undermines the assumption that human review will catch the gaps. Self-reported confidence is not a useful proxy for actual correctness in this context, though the study was not designed to test what a better proxy might look like.

C. Where the Evidence is Thin

Several gaps in the current literature are worth flagging. Most empirical work in the corpus is Python-centric and Copilot-centric. Long-term effects of LLM use on code quality, after release rather than at merge time, are not covered by any paper in the corpus. Non-determinism is addressed by only a few studies. Work that closes any of these gaps would be genuinely useful, though the Python-and-Copilot bias is partly a data availability problem.

VI. THREATS TO VALIDITY

Construct validity. “Functional correctness” is measured differently across the studies (pass@1, pass@k, test-pass rates on student tasks, human-judged correctness, qualitative observation). When synthesizing across them we treat these as the same construct, which they are not entirely.

Internal validity. Only a single screener was used. A second screener could have disagreed with some inclusion and exclusion decisions. We mitigated this by stating the criteria explicitly in Section III, but the solo-screener limitation is real. The screening was also iterative rather than tracked with explicit counts, as discussed in Section III; this limits the replicability of the selection process.

External validity. The corpus is heavily Python-centric and heavily Copilot-centric. Conclusions probably generalize to similar models and similar tasks but not beyond. The field is also moving quickly, and some early papers evaluate model generations that are already behind the state of the art.

Recency and small corpus. The corpus is intentionally small (eleven primary studies) and includes one short paper from 2022. A larger corpus might reveal patterns that this review misses. The tradeoff was depth of reading against breadth of coverage, and we chose depth.

VII. CONCLUSION

This paper synthesized recent empirical evidence on the functional correctness of code produced with LLM assistance. Whether LLM assistance improves correctness turns out to be the wrong question to ask. What the evidence shows instead is something more conditional: correctness shifts in distribution, not uniformly. On simple benchmarks with weak tests, LLM-generated code is usually correct. On harder benchmarks, class-level tasks, and realistic repository-level issues, correctness drops sharply. Comparisons with non-LLM development

show productivity gains that are more consistent than correctness gains, and at least one controlled study shows users becoming more confident while producing less correct code. Task difficulty and dependency complexity are the strongest factors, prompt formulation has a smaller effect than often claimed, and evaluation methodology — test-suite strength, generation strategy, non-determinism — can shift measured correctness by tens of percentage points.

LLM assistance helps more on simple tasks and less on complex ones, and the mistakes it makes are disproportionately hard for reviewers to notice. Future work that measures correctness after release, across more languages, and under realistic project context would be the most useful next step.

ACKNOWLEDGMENTS

We used Claude Sonnet (Anthropic) for language polishing, grammar correction, and improving clarity of expression. All intellectual contributions, including the analysis, synthesis, arguments, and conclusions, are solely the author’s own.

REFERENCES

- [1] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, “Asleep at the Keyboard? Assessing the Security of GitHub Copilot’s Code Contributions,” in *Proc. IEEE Symp. Security and Privacy (S&P)*, 2022, pp. 754–768, doi: 10.1109/SP46214.2022.9833571.
- [2] M. Chen et al., “Evaluating Large Language Models Trained on Code,” arXiv preprint, 2021. [Online]. Available: <https://arxiv.org/abs/2107.03374>
- [3] B. Kitchenham, L. Madeyski, and D. Budgen, “SEGRESS: Software Engineering Guidelines for REporting Secondary Studies,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 3, pp. 1273–1298, 2023, doi: 10.1109/TSE.2022.3174092.
- [4] M. V. Mäntylä, “Mini-SLR Guide for Thesis and Seminar Work,” University of Helsinki, 2024. [Online]. Available: <https://mmantyla.github.io/mini-slr-for-thesis/>
- [5] J. Liu, C. S. Xia, Y. Wang, and L. Zhang, “Is Your Code Generated by ChatGPT Really Correct? Rigorous Evaluation of Large Language Models for Code Generation,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. [Online]. Available: <https://arxiv.org/abs/2305.01210>
- [6] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “SWE-bench: Can Language Models Resolve Real-World GitHub Issues?,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2024. [Online]. Available: <https://arxiv.org/abs/2310.06770>
- [7] T. Y. Zhuo et al., “BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions,” in *Proc. Int. Conf. Learning Representations (ICLR)*, 2025. [Online]. Available: <https://arxiv.org/abs/2406.15877>
- [8] X. Du, M. Liu, K. Wang, H. Wang, J. Liu, Y. Chen, J. Feng, C. Sha, X. Peng, and Y. Lou, “Evaluating Large Language Models in Class-Level Code Generation,” in *Proc. 46th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, 2024, doi: 10.1145/3597503.3639219.
- [9] F. Cassano, J. Gouwar, D. Nguyen, S. Nguyen, L. Phipps-Costin, D. Pinckney, M.-H. Yee, Y. Zi, C. J. Anderson, M. Q. Feldman, A. Guha, M. Greenberg, and A. Jangda, “MultiPL-E: A Scalable and Polyglot Approach to Benchmarking Neural Code Generation,” *IEEE Trans. Softw. Eng.*, vol. 49, no. 7, pp. 3675–3691, 2023, doi: 10.1109/TSE.2023.3267446.
- [10] A. Mastropaolo, L. Pascarella, E. Guglielmi, M. Ciniselli, S. Scalabrino, R. Oliveto, and G. Bavota, “On the Robustness of Code Generation Techniques: An Empirical Study on GitHub Copilot,” in *Proc. 45th IEEE/ACM Int. Conf. Softw. Eng. (ICSE)*, 2023, pp. 2149–2160, doi: 10.1109/ICSE48619.2023.00181.
- [11] A. Moradi Dakhel, V. Majdinasab, A. Nikanjam, F. Khomh, M. C. Desmarais, and Z. M. J. Jiang, “GitHub Copilot AI pair programmer: Asset or Liability?,” *J. Syst. Softw.*, vol. 203, Art. 111734, 2023, doi: 10.1016/j.jss.2023.111734.
- [12] N. Perry, M. Srivastava, D. Kumar, and D. Boneh, “Do Users Write More Insecure Code with AI Assistants?,” in *Proc. ACM SIGSAC Conf. Computer and Communications Security (CCS)*, 2023, doi: 10.1145/3576915.3623157.
- [13] S. Barke, M. B. James, and N. Polikarpova, “Grounded Copilot: How Programmers Interact with Code-Generating Models,” *Proc. ACM Program. Lang.*, vol. 7, OOPSLA1, Art. 78, 2023, doi: 10.1145/3586030.
- [14] N. Nguyen and S. Nadi, “An Empirical Evaluation of GitHub Copilot’s Code Suggestions,” in *Proc. 19th Int. Conf. Mining Software Repositories (MSR)*, 2022, pp. 1–5, doi: 10.1145/3524842.3528470.
- [15] R. Khojah, F. Gomes de Oliveira Neto, M. Mohamad, and P. Leitner, “The Impact of Prompt Programming on Function-Level Code Generation,” *IEEE Trans. Softw. Eng.*, vol. 51, no. 8, pp. 2381–2395, 2025, doi: 10.1109/TSE.2025.3587794.